

MongoDB → PostgreSQL / MySQL

Database Migration — Practical Concept Guide

Yes, MongoDB nunchi PostgreSQL or MySQL ki future lo migrate avvachu. Kaani idi direct copy operation kaadu. Main work document-oriented data model ni relational structure ki transform cheyyadam.

1. Migration Possible aa?

MongoDB → PostgreSQL and MongoDB → MySQL rendu possible. Typical approach: **Extract → Transform → Load (ETL)**.

MongoDB → Extract → JSON / ETL → Transform → Relational Schema → PostgreSQL / MySQL

2. Why Direct Copy Kaadu?

MongoDB lo documents, nested objects and arrays flexible ga store cheyyachu. PostgreSQL/MySQL lo tables, rows, columns, primary keys and foreign keys use chestaru.

MongoDB example

```
users
{ "_id": "101", "name": "Santhosh", "courses": [{"course_id": "C101", "name": "DevOps"}] }
```

Possible PostgreSQL design

```
users → id, name
courses → id, name
user_courses → user_id, course_id
```

Ante MongoDB lo embedded array ga unna data PostgreSQL lo separate tables and relationships ga model cheyyalsi ravachu.

3. Migration Steps

Extract

- MongoDB collections/documents nunchi data read cheyyali.
- Sharding unte multiple shards nunchi data collect cheyyali.

Transform

- Documents ni target relational tables ki map cheyyali.
- Nested objects/arrays ni separate tables or JSON/JSONB columns ga design cheyyachu.
- IDs, dates, numeric types and null handling ni target format ki convert cheyyali.

Load

- Transformed data ni PostgreSQL/MySQL tables lo load cheyyali.
- Primary keys, foreign keys, indexes and constraints validate cheyyali.

Validate

- Source and target record counts compare cheyyali.
- Users, orders, payments, subscriptions lanti critical data verify cheyyali.
- Application queries and transactions test cheyyali.

4. Replica vs Sharding vs Scaling

Concept	Purpose
Replica Set	Same data multiple nodes lo maintain cheyyadam; mainly high availability / redundancy.
Sharding	Different portions of data ni multiple shards/nodes ki distribute cheyyadam; horizontal scaling kosam.
Vertical Scaling	Same machine ni bigger CPU/RAM/storage configuration ki scale cheyyadam.
Horizontal Scaling	More machines/nodes add chesi workload or data ni distribute cheyyadam.

Memory: Replica = same data copies. Shard = different data distribution.

5. PostgreSQL vs MySQL as Migration Targets

Area	PostgreSQL	MySQL
Relational model	Strong relational model with foreign keys and transactions.	Strong relational model with foreign keys and transactions.
Flexible data	JSONB can store flexible/document-style data inside relational tables.	JSON support is available for document-style data.
MongoDB migration	Possible through schema mapping and ETL; good when relationships and flexible data both matter.	Possible through schema mapping and ETL; good for conventional relational workloads.
EduProva fit	Users, enrollments, payments, subscriptions + flexible content fit well.	Also possible for relational workloads.

6. Designing Today for a Future Migration

Database-specific access ni application business logic nunchi separate cheyyadam useful.

Today: Application → Repository / Data Access Layer → MongoDB

Future: Application → Same Repository / Data Access Layer → PostgreSQL

- MongoDB-specific queries and document assumptions still need redesign where they do not map cleanly to SQL.

7. Cost & Complexity

Factor	Migration impact
Data volume	More data means longer extraction, transformation and loading.
Sharding	Distributed data makes extraction and validation more involved.
Nested documents	May need separate relational tables or JSON/JSONB columns.
Indexes	MongoDB indexes need review and redesign for the target database.
Downtime	Near-zero downtime needs additional replication/change-data-capture planning.
Application queries	MongoDB query patterns may need to be rewritten as SQL.

8. EduProva Example

Suppose EduProva starts with MongoDB and later decides that payments, subscriptions, enrollments and reporting are better served by PostgreSQL.

Current

EKS / Application → MongoDB Atlas → MongoDB collections

Future

EKS / Application → Aurora PostgreSQL → Relational tables + JSONB

Migration activities

- Design PostgreSQL schema.
- Map MongoDB collections/documents to tables and JSONB where appropriate.
- Migrate historical data.
- Validate counts, relationships and financial records.
- Update application queries/configuration.
- Test performance, transactions and reporting.
- Plan cutover and rollback.

9. Interview Answer

“Yes, MongoDB can be migrated to PostgreSQL or MySQL in the future. However, it is not a direct cluster-to-database copy. We need to transform the document-oriented data model into a relational schema, handle relationships and constraints, and migrate the data using an ETL or migration process. If future migration is a possibility, I would isolate database-specific logic behind a data-access layer to reduce application coupling.”

10. Quick Revision

Question	Answer
MongoDB → PostgreSQL?	Yes; schema mapping and ETL are normally required.
MongoDB → MySQL?	Yes; schema mapping and ETL are normally required.
Direct copy?	No. Document structure may need relational redesign.
Horizontal scaling?	More machines/nodes; MongoDB sharding is the main data-distribution mechanism.
Replica = Shard?	No. Replica mainly provides redundancy; shard distributes data/workload.
Vertical scaling?	Make the same machine bigger.
Future migration best practice?	Keep database-specific access behind a repository/data-access layer.

Final takeaway

MongoDB → PostgreSQL/MySQL is absolutely possible. The real challenge is not copying the servers; it is transforming the data model, queries, indexes and application integration to fit the relational database.